FlashKDA

A chunkwise CUTLASS kernel for Kimi Delta Attention

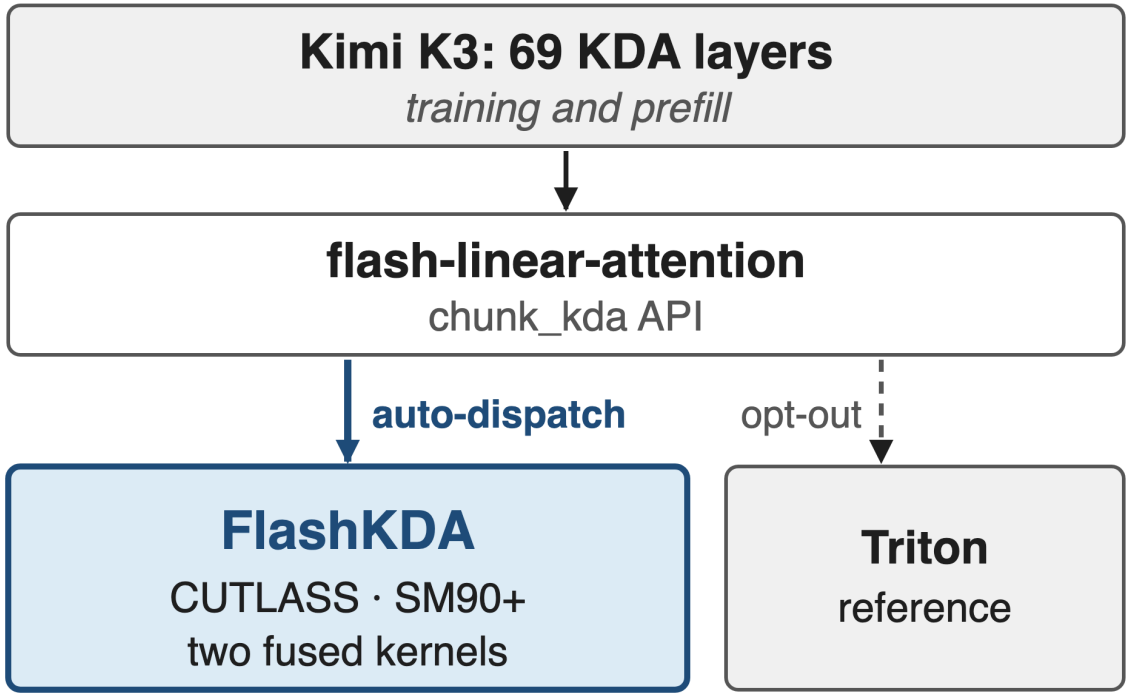
Moonshot AI · Kimi K3 (open-source FlashKDA v1)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Attention Kernels · arxiv.org/abs/2607.24653

Every KDA layer in K3 runs through FlashKDA

- K3: 69 KDA layers, training and prefill
- Auto-dispatched behind FLA's `chunk_kda`
- KDA math (state update, chunks): deck 05



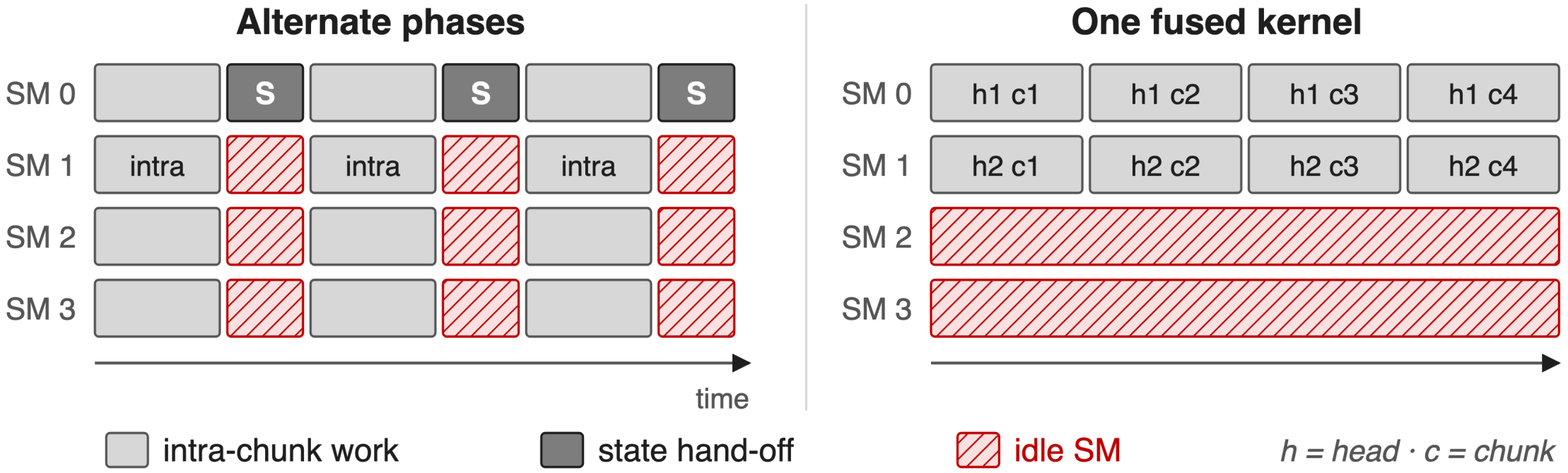
The reference KDA kernel trails simpler GDN

4.84 ms

Triton chunk_kda forward, one 8K-token sequence × 96 heads on H20:
about 1.5× the scalar-gate GDN kernel (3.20 ms)

FlashKDA BENCHMARK_H20.md, T=8192, H=96, D=128, fixed length; 1.5× = 4.84 / 3.20 (derived)

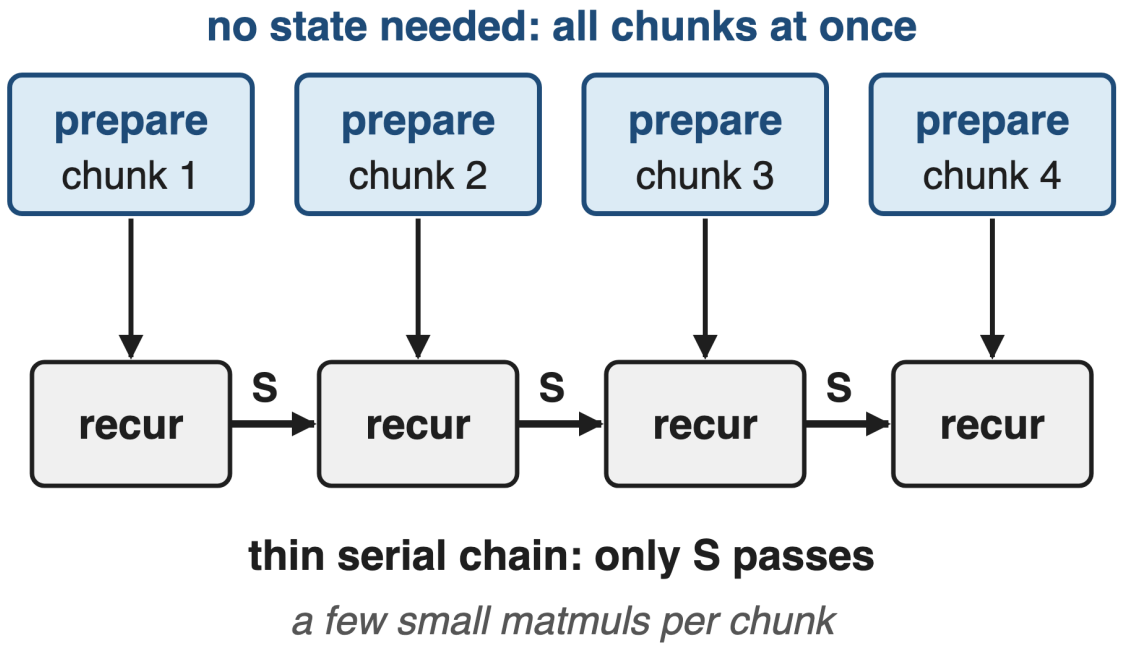
Serial state hand-offs leave SMs idle



The recurrence offers only sequences × heads CTAs; the rest could use tiles × heads

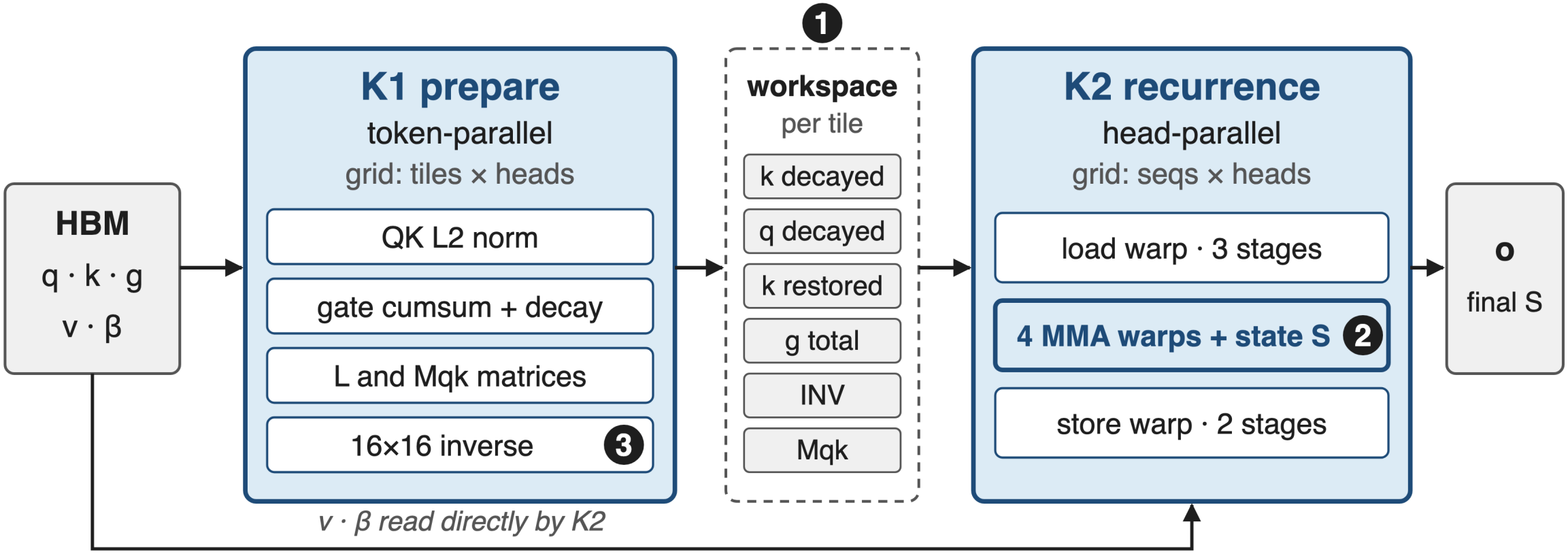
Insight: only the state has to wait

- Decay, L2 norm, inverse, masked QK^T: no S needed
- Only a few small matmuls touch S



Prepare every chunk in parallel; leave a thin serial chain

FlashKDA: prepare in parallel, then recur

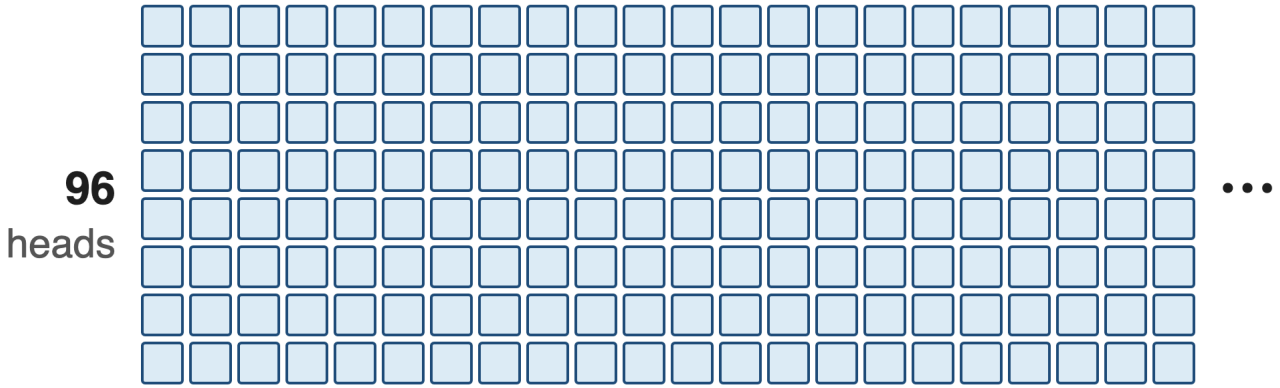


❶ split by parallelism · ❷ warp-specialized recurrence · ❸ 16-token chunks (FlashKDA v1 source, csrc/smxx)

❶ Split the kernel along its parallelism

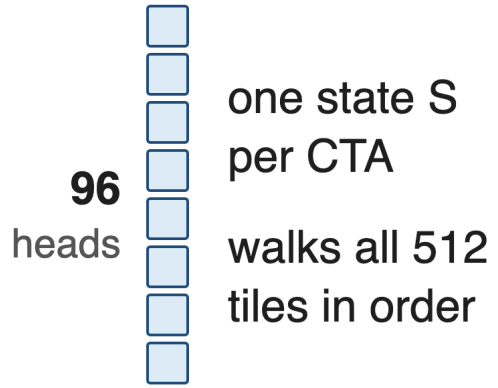
example: one 8K-token sequence with 96 heads

K1 prepare: tiles × heads



49,152 CTAs

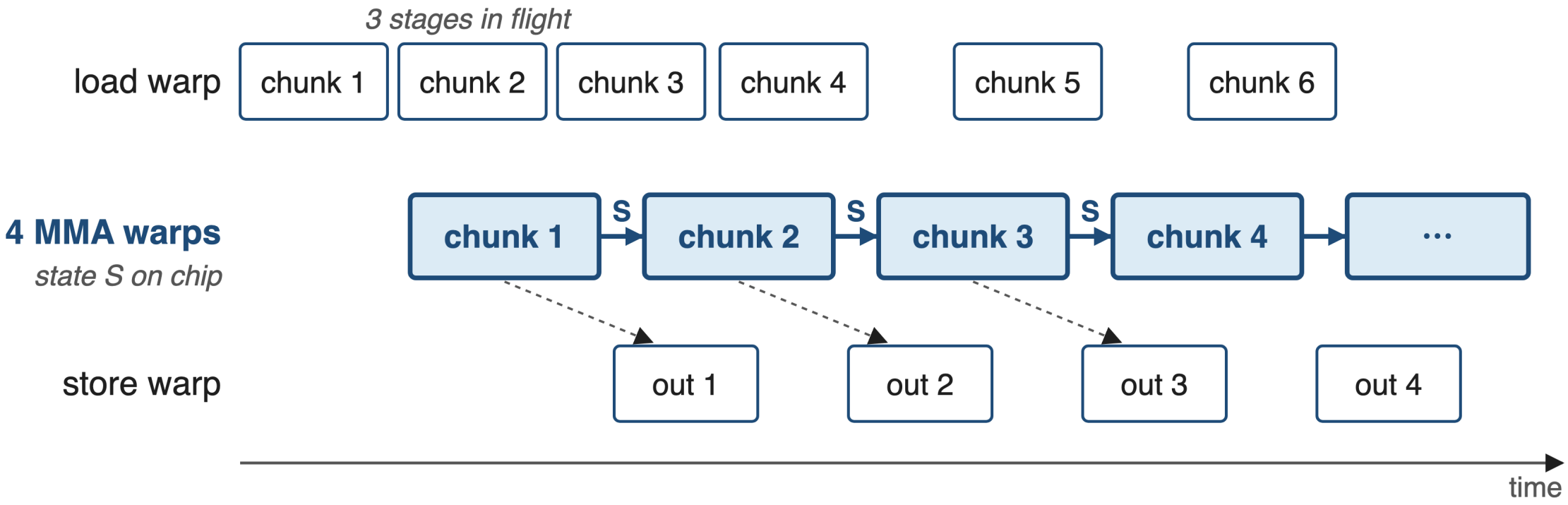
K2 recurrence: seqs × heads



96 CTAs

Two kernels instead of one fused kernel: at least 15% faster end to end

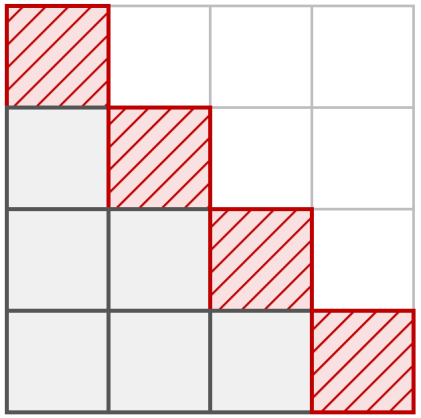
❷ Warps overlap memory with the state chain



Per chunk: k·S, q·S → U = INV·β·(v – k·S) → o = q·S + Mqk·U → S ← γ·S + k^TU

❸ 16-token chunks make the inverse cheap

64-token chunk (FLA)



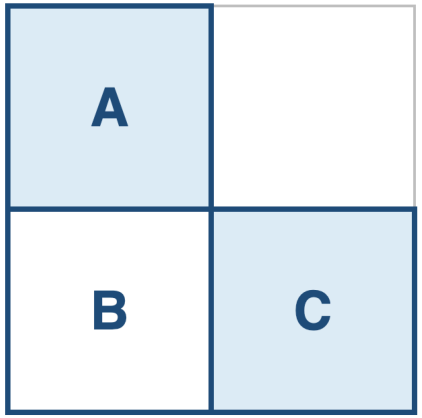
16-token sub-chunks
rescaled in log space

64×64 triangular inverse

g ≥ –5

16 tokens fit
in BF16 range

16-token chunk (FlashKDA)



I + L (16×16)

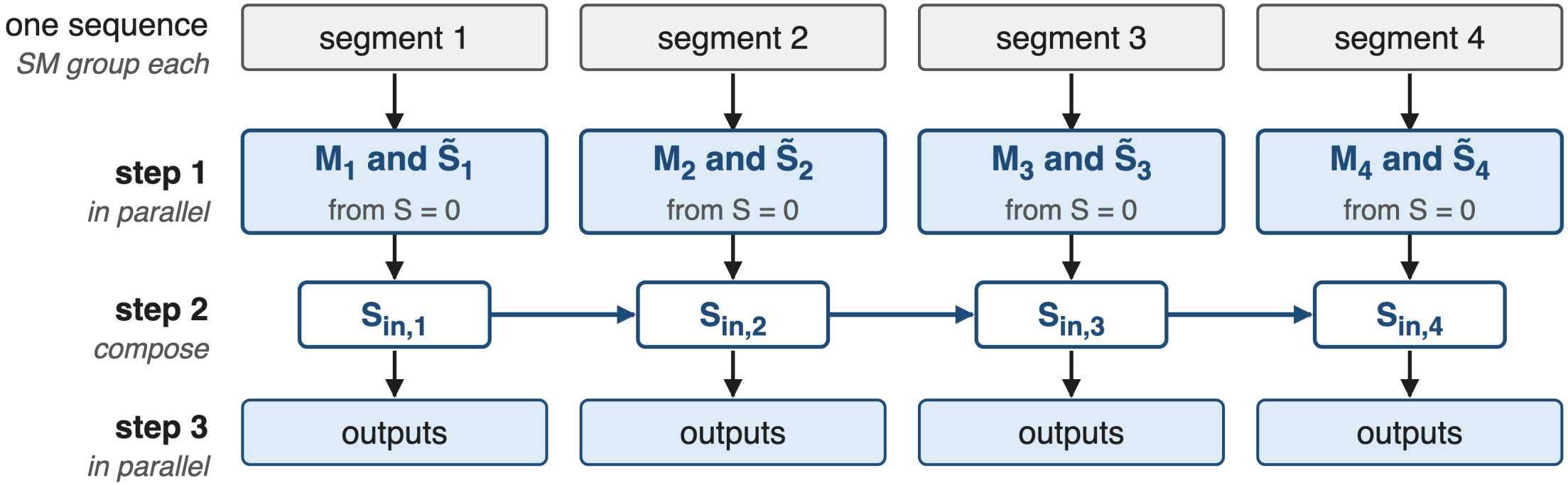
A, C: 8×8 blocks
FP32 forward
substitution

B: merged by two
BF16 HMMA GEMMs

off-diagonal block of the inverse = $-C^{-1} \cdot B \cdot A^{-1}$

Bounded decay (g ≥ –5, deck 05) is what makes the small chunk numerically safe

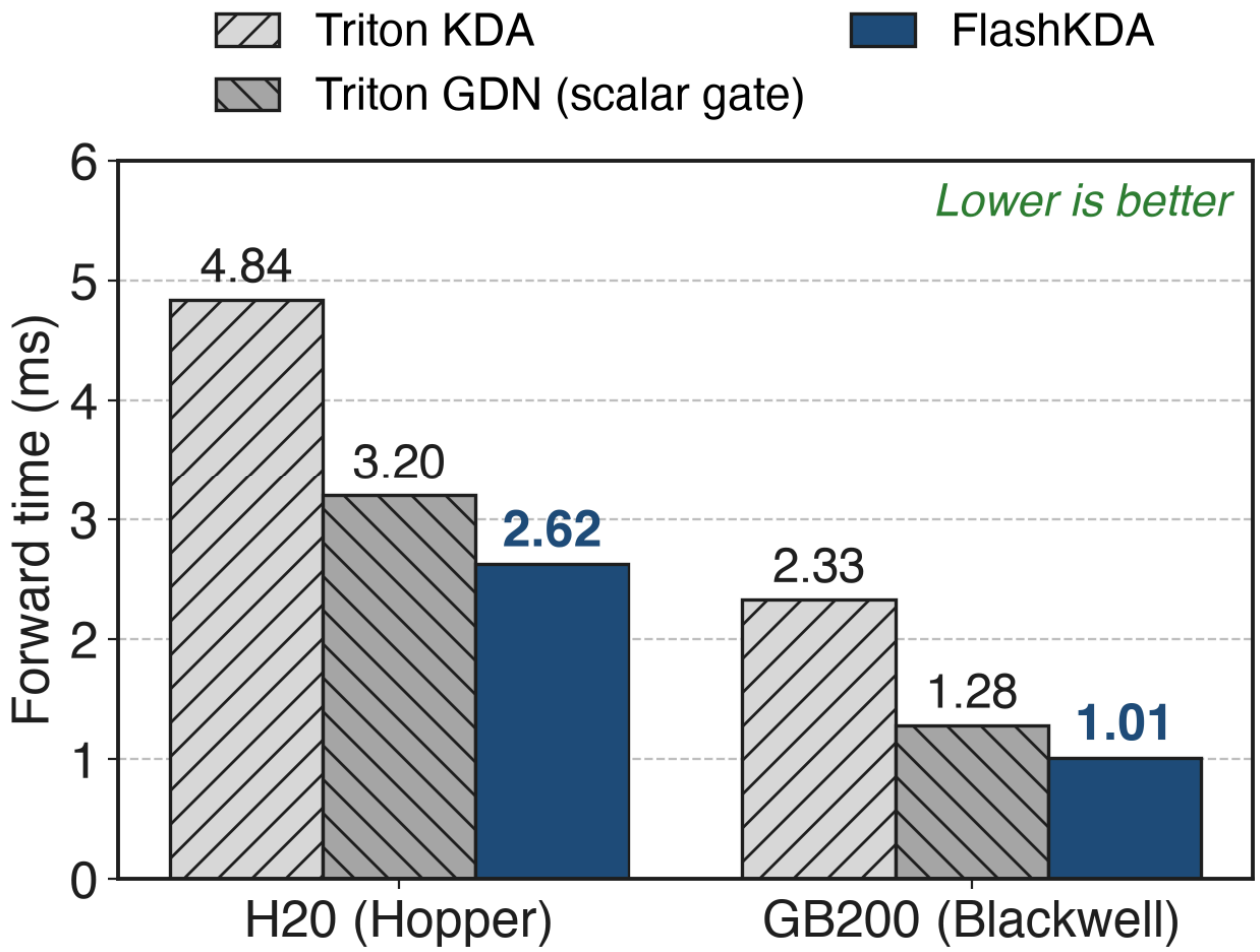
Long prefill also splits one sequence across SMs



The same decomposition across GPUs is KDA Context Parallelism: deck 69

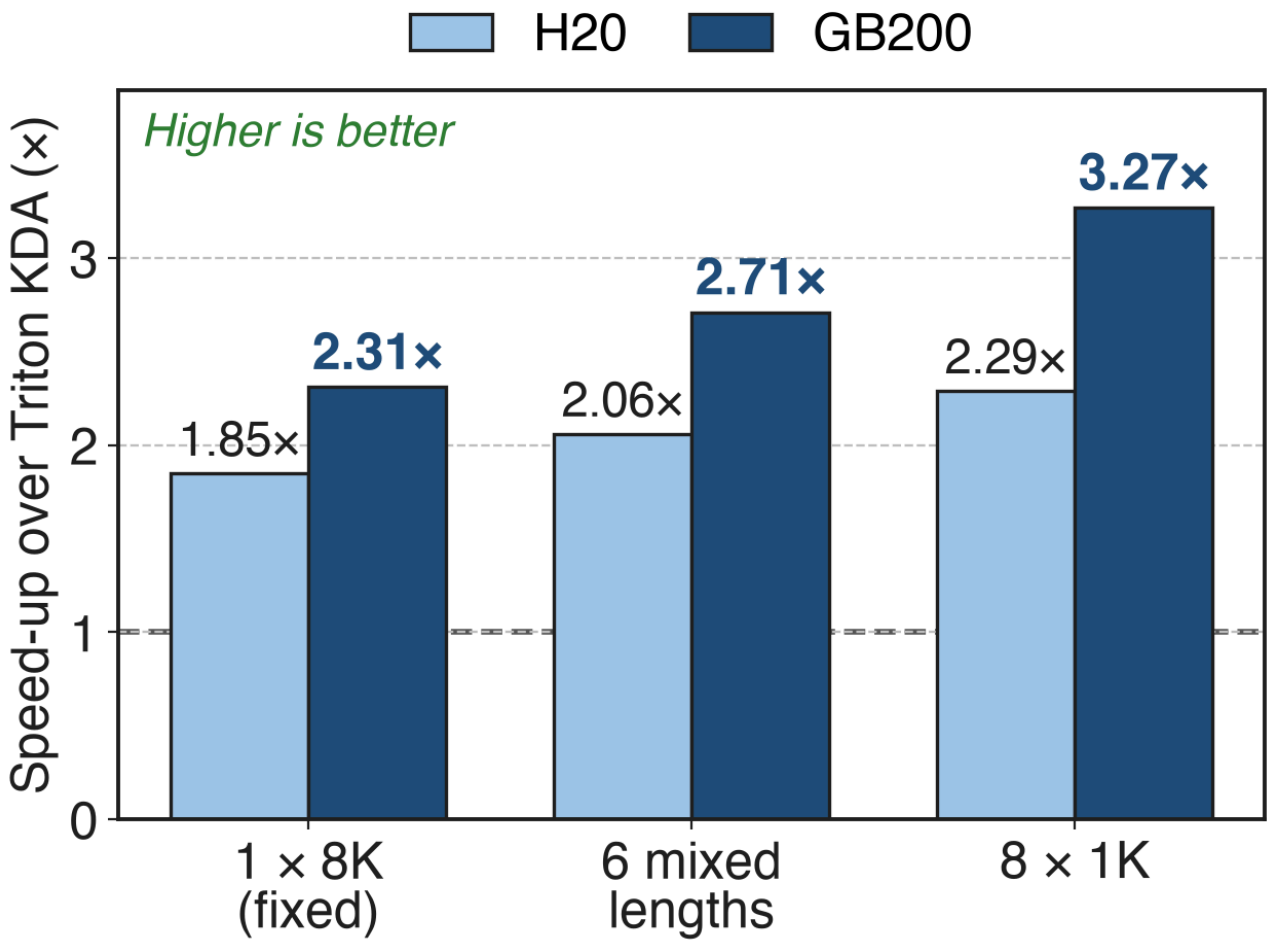
1.85-2.31× faster than the Triton kernel

- One 8K-token sequence, 96 heads, head dim 128
- Also beats scalar-gate GDN: **1.22×** H20, **1.27×** GB200
- The per-channel gate no longer costs time



Variable-length batches gain up to 3.27×

- Same 8K tokens, split into several sequences
- Triton time stays flat; FlashKDA's drops
- 64 heads: 1.70-3.21× on GB200



Costs: Hopper+, fixed head size, forward only

In FlashKDA v1

- ✓ **Hopper and Blackwell**
  SM90+ · CUDA 12.9+
- ✓ **Head dims K = V = 128**
  only this shape
- ✓ **Bounded gate**
  lower bound in [–5, 0]
- ✓ **Forward pass and varlen**
  prefill · cu_seqlens batches

Not in the public v1

- ✗ **Backward pass**
  K3 reports training use
- ✗ **Decoding**
  separate fused kernel (K3 §5.4.2)
- ✗ **Other head dims**
  and pre-Hopper GPUs
- ✗ **Unbounded gates**
  the API requires a lower bound

One regression: 64 heads, fixed length on GB200 runs at 0.96× of GDN

Summary

- Triton KDA: **4.84 ms** per 8K × 96-head forward on H20, 1.5× GDN
- **FlashKDA**: prepare all chunks in parallel, carry only S serially
- **1.85×** (H20) to **3.27×** (GB200, varlen) faster; beats GDN
- Previous: KDA math, deck 05 · Next: cross-GPU KCP, deck 69

https://arxiv.org/abs/2607.24653